

Robust QR-DQN Implementation for CleanRL

Luxin Matasaru¹

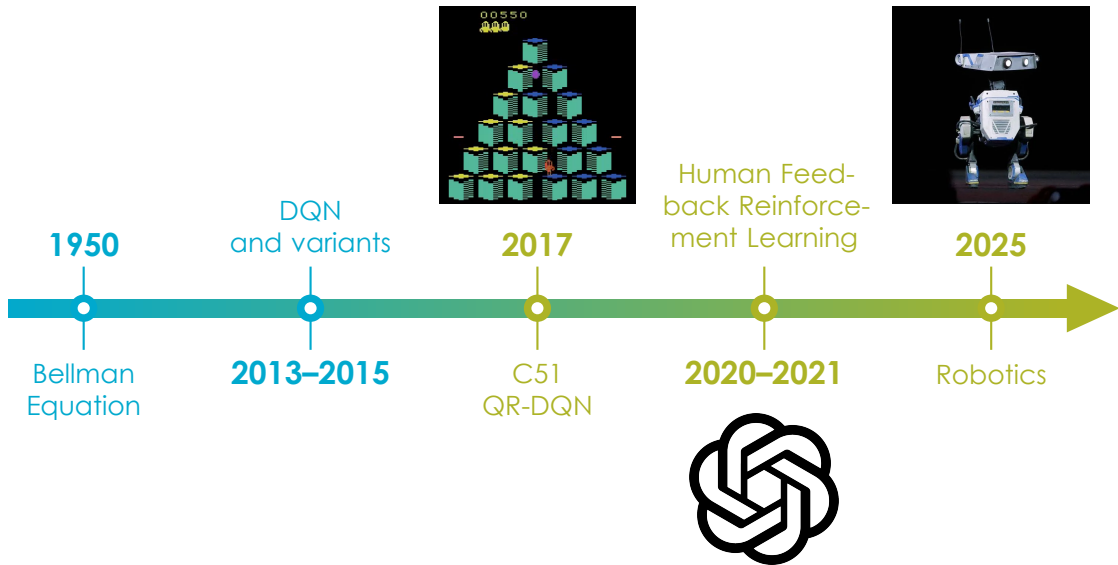
Supervisor: Simo Alami²

¹École polytechnique

²LIX, École polytechnique

Outline

- 1 Introduction
- 2 Reinforcement learning
- 3 Results



Motivation

Stable Baselines, Dopamine, ACME etc.

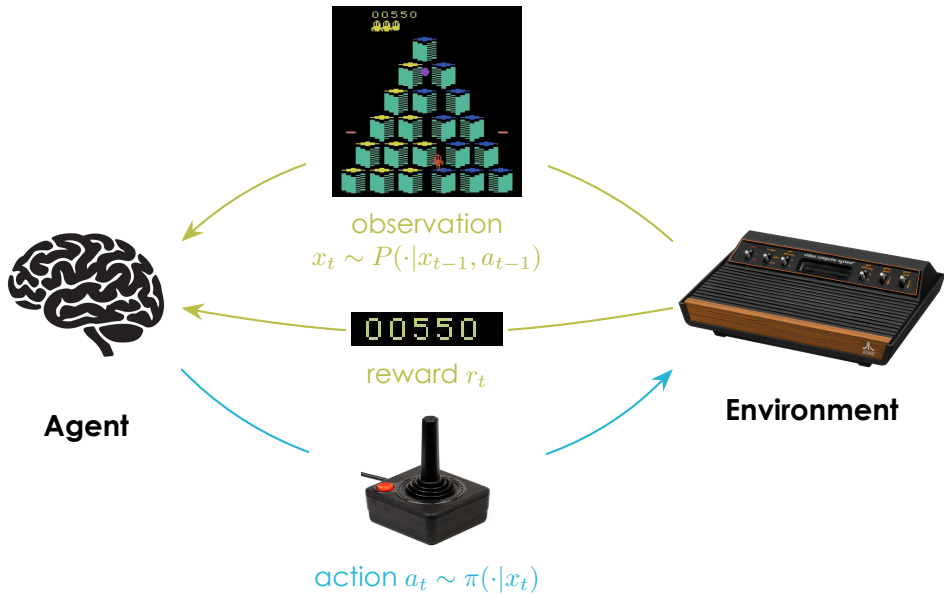
- Modular
- Complete implementations with all the bells and whistles
- Hard to pick up and modify

CleanRL

- Single file implementations
- Readable code, full details visible
- Easy benchmarking and reproducibility

We noticed that QRDQN was missing from CleanRL

The reinforcement learning problem



Main approaches

- **Supervised learning** requires labeled data - impossible to gather in most settings.
- **Unsupervised learning** learns patterns in data, but has no concept of actions or policy.
- Agent needs to learn from prior experience - **reinforcement learning!**

Main approaches

Define the Q-function

$$Q(x, a) = \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t R_t(x_t, a_t) \right]$$

It follows the **Bellman equation**

$$Q(x, a) = \mathbb{E}[R(x, a)] + \gamma \mathbb{E}_{P, \pi}[Q(x', a')] \\ x' \sim P(\cdot | x, a), \quad a' \sim \pi(\cdot | x')$$

An agent maximizing $Q(x, a)$ will have to take the action a' which gives the best $Q(x', a')$.

An optimal agent is **greedy**!

A natural approach is to estimate / "learn" $Q(x', a')$.

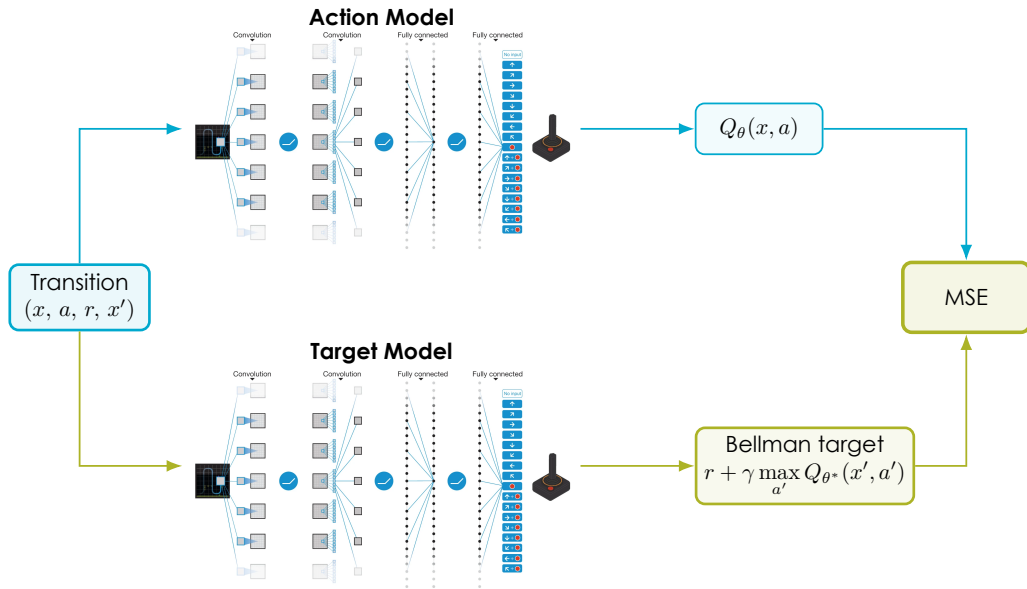
From the Bellman equation, we can define the operator $\mathcal{T}^\pi$

$$\mathcal{T}^\pi Q(x, a) := \mathbb{E}[R(x, a)] + \gamma \mathbb{E}_{P, \pi}[Q(x', a')]$$

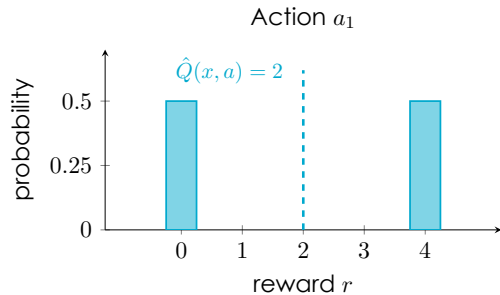
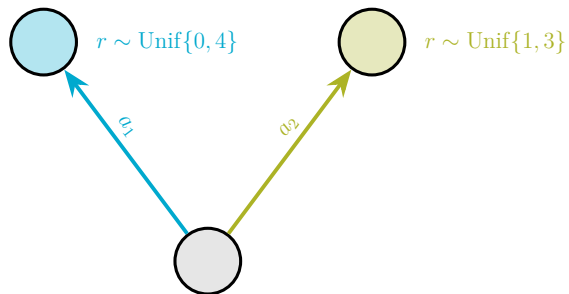
$\mathcal{T}^\pi$ is a contraction in $\mathbb{R}^{\mathcal{X} \times \mathcal{A}} \implies$ it has a unique fixed point.

How to find it?

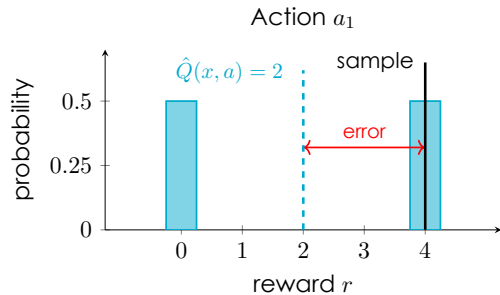
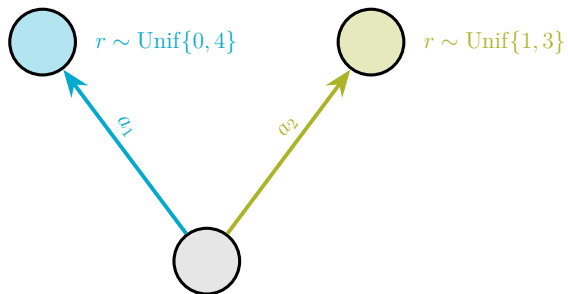
- Starting with Q_0 apply $\mathcal{T}^\pi$ repeatedly $\implies$ value iteration
- Approximate Q with neural networks. Minimize error (MSE) between $Q(x, a)$ and $\mathcal{T}^\pi Q(x, a) \implies$ DQN (et al., 2015)



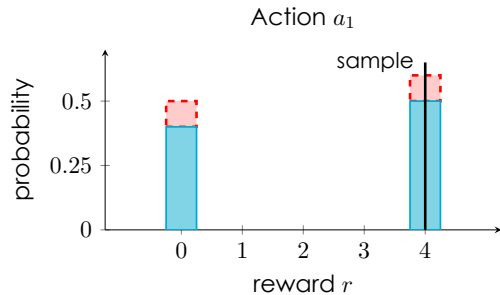
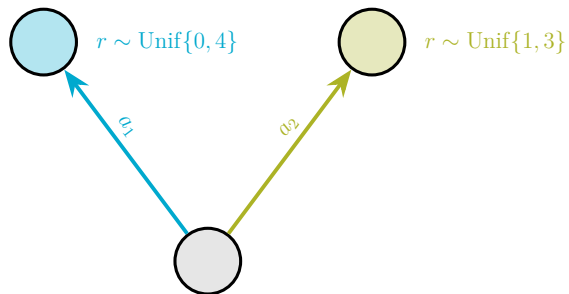
Distributional RL



Distributional RL



Distributional RL



Distributional RL

- Model the distribution of the return $Z(x, a) = \sum_{t=0}^{\infty} \gamma^t R_t(x_t, a_t)$.
- $Q(x, a)$ is then just the expected value $\mathbf{E}[Z(x, a)]$.

How?

- The distributional Bellman operator $\mathcal{T}^{\pi} Z(x, a) \stackrel{D}{=} R(x, a) + \gamma Z(x', a')$
- Minimize some "distance" between $\mathcal{T}^{\pi} Z(x, a)$ and $Z(x, a)$.

KL divergence and **Wasserstein distance** are popular choices, however neither work directly.

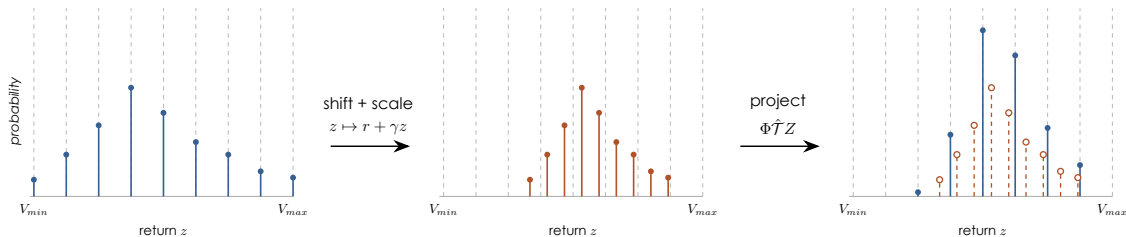
Distributional RL - C51 (Bellemare et al., 2017)

Fix **support**, predict **probabilities**.

- Model $Z(x, a)$ using $Z_\theta = \sum p_i(x, a) \delta_{z_i}$ with

$$V_{min} = z_0 < z_1 < \dots < z_{N-1} = V_{max}$$

- Project $\hat{T}Z_\theta(x, a)$ onto the support.
- Loss function given by the cross entropy term of $KL(\Phi \hat{T}Z_{\hat{\theta}}(x, a) \| Z_\theta(x, a))$.



Distributional RL - QRDQN (Dabney et al., 2017)

Fix **probabilities**, predict **support**.

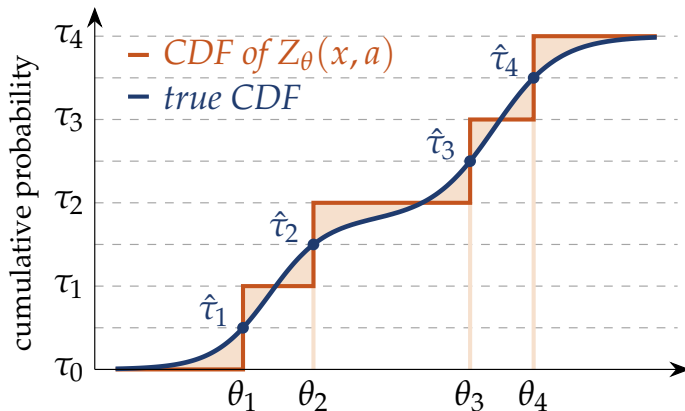
- Define $Z_\theta(x, a)$ as

$$Z_\theta(x, a) := \frac{1}{N} \sum_{i=1}^N \delta_{\theta_i(x, a)}$$

- The $Z_\theta(x, a)$ which minimizes $W_1(Z(x, a), Z_\theta(x, a))$ has

$$\theta_i(x, a) = F_{Z(x, a)}^{-1}(\hat{\tau}_i)$$

- Train a neural network with input x and output $|\mathcal{A}| \times N$, the $F_{Z(x, a)}^{-1}(\hat{\tau}_i)$ for all i and a .



Atari 2600



Battle Zone

Double Dunk

Name this Game

Phoenix

Q*bert

We evaluate on 5 of the 57 Atari 2600 games.

Aitchison et al. (2022) suggest performance on this subset is representative for the entire set.

Results on Atari-5

Game	DQN	C51	QR-DQN-0	QR-DQN-1	Ours
Battle Zone	29,900	28,742	35,580	39,268	49,500
Double Dunk	-6.6	2.5	12.3	21.9	18.8
Name this Game	8,207.8	12,542	17,557	21,890	23,013
Phoenix	8,485.2	17,490	65,767	16,585	124,298
Q*Bert	13,117.3	23,784	26,946	572,510	26,822

In order to compare our results with Dabney et al. (2017) we report **best agent performance**:

- Every 1M steps in training, evaluate over 10 episodes and save the mean.
- Report the best such score.

References

- Matthew Aitchison, Penny Sweetser, and Marcus Hutter. Atari-5: Distilling the arcade learning environment down to five games, 2022. URL <https://arxiv.org/abs/2210.02019>.
- Marc G. Bellemare, Will Dabney, and Rémi Munos. A distributional perspective on reinforcement learning, 2017. URL <https://arxiv.org/abs/1707.06887>.
- Will Dabney, Mark Rowland, Marc G. Bellemare, and Rémi Munos. Distributional reinforcement learning with quantile regression, 2017. URL <https://arxiv.org/abs/1710.10044>.
- Volodymyr Mnih et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015. doi: 10.1038/nature14236. URL <https://doi.org/10.1038/nature14236>.

Appendix - Quantile Huber loss

Quantile regression is a method for approximating the quantile function of a distribution.

Adapt the quantile regression loss to be smooth around 0 $\implies$ the **quantile Huber loss**:

$$\rho_{\tau}^{\kappa}(u) = |\tau - \delta_{u < 0}| \mathcal{L}_{\kappa}(u)$$
$$\mathcal{L}_{\kappa}(u) = \begin{cases} \frac{1}{2}u^2 & \text{if } |u| \leq \kappa \\ \kappa \left(|u| - \frac{1}{2}\kappa \right) & \text{otherwise} \end{cases}$$

Algorithm 1: Quantile Regression Q-Learning (Algorithm 1 Dabney et al. (2017))

Input: $x, a, r, x', \gamma \in [0, 1)$

Data: N, κ

// Compute distributional
Bellman target

$Q(x', a') \leftarrow \sum_j q_j \theta_j(x', a');$

$a^* \leftarrow \arg \max_{a'} Q(x, a');$

$\mathcal{T}\theta_j \leftarrow r + \gamma \theta_j(x', a^*), \quad \forall j;$

// Compute quantile regression
loss

Output: $\sum_{i=1}^N \mathbb{E}_j \left[\rho_{\hat{\tau}_i}^{\kappa}(\mathcal{T}\theta_j - \theta_i(x, a)) \right]$

Appendix - Distribution

Step 742

